

The 3-Sum Report

Said Hossain Reza

Abstract

Denna rapport presenterar en undersökning av 3 summa problemet, vilket innebär att hitta alla unika tripplar i en lista med heltal som summerar till ett givet mål. Vi implementerade en Brute force-algoritm och utvärderades dess prestanda genom experiment. Rapporten diskuterar problemformulering, implementering, experimentupplägg och resultat. Dessutom uppskattade vi tidskomplexiteten för algoritmen med hjälp av en log-log metod och linjär regression.

Introduktion

Den här rapporten fokuserar på 3 summaproblemet och utvärderar brute force-metoden för att lösa det. den experimentella processen inkluderade mätning av exekveringstider för olika inmatningsstorlekar och verifiering av resultatens överensstämmelse.

Problemformulering

3-sum problem:

Givet en lista med heltal $n_1, n_2, \dots, n_p$ och ett målheltal s , hitta alla unika tripplar (n_i, n_j, n_k) där $i \neq j \neq k$, $n_i + n_j + n_k = s$, med villkoret att $n_i \leq n_j \leq n_k$

Returnerar:

En lista över unika tripplar (n_i, n_j, n_k) sorterade så att $n_i \leq n_j \leq n_k$.

Denna formulering säkerställer att det inte finns några dubbla tripplar till exempel $(5, 7, 8)$ och $(7, 5, 8)$ räknas inte som olika.

Exempel 1

Given input:

Lista: $[-1, 0, 1, 2, -1, -4,]$

Target: 0:

Output: $[(-1, -1, 2), (-1, 0, 1)]$

Explanation:

$(-1, -1, 2): -1 + -1 + 2 = 0$, $(-1, 0, 1): -1 + 0 + 1 = 0$

Example 2:

List: [3, -2, -1, 0, 2, -3, 1, 4, -4, 5]

Target: 0

Output: [(-4, -1, 5), (-4, 0, 4), (-4, 1, 3), (-3, -2, 5), (-3, 0, 3), (-3, 1, 2), (-2, -1, 3), (-2, 0, 2), (-1, 0, 1)]

Explanation:

$(-4, -1, 5): -4 + -1 + 5 = 0$

$(-4, 0, 4): -4 + 0 + 4 = 0$

$(-2, 0, 2): -2 + 0 + 2 = 0$

Etc.....

Brute Force solution

Brute force-metoden itererar genom alla möjliga tripplar i inmatningslistan för att identifiera kombinationer som summerar till målvärdet. För varje kombination kontrollerar algoritmen om summan av tre element är lika med målet. Genom att använda en set struktur säkerställs att varje unik trippel lagras endast en gång, och varje trippel sorteras för att undvika permutationer som räknas som dubletter (till exempel (-1, 0, 1) och (0, -1, 1))

Python implementering

Python implementationen som används för experimenten visas i figur 1.

```
def threesum_brute(lst, target=0):  
    unique_triples = set()  
    n = len(lst)  
    for i in range(n-2):  
        for j in range(i+1, n-1):  
            for k in range(j+1, n):  
                if lst[i] + lst[j] + lst[k] == target:
```

```
triple = tuple(sorted([lst[i], lst[j], lst[k]]))  
  
unique_triples.add(triple)  
  
return list(unique_triples)
```

Observations:

Resultaten är konsekventa mellan körningarna, med en tydlig trend där körningstiden ökar som förväntat för en algoritm med tidskomplexitet $O(n^3)$. Algoritmens långsamhet märks särskilt vid större inmatningsstorlekar, men resultaten är stabila och visar inga avvikelser mellan körningarna.

Experimental setup:

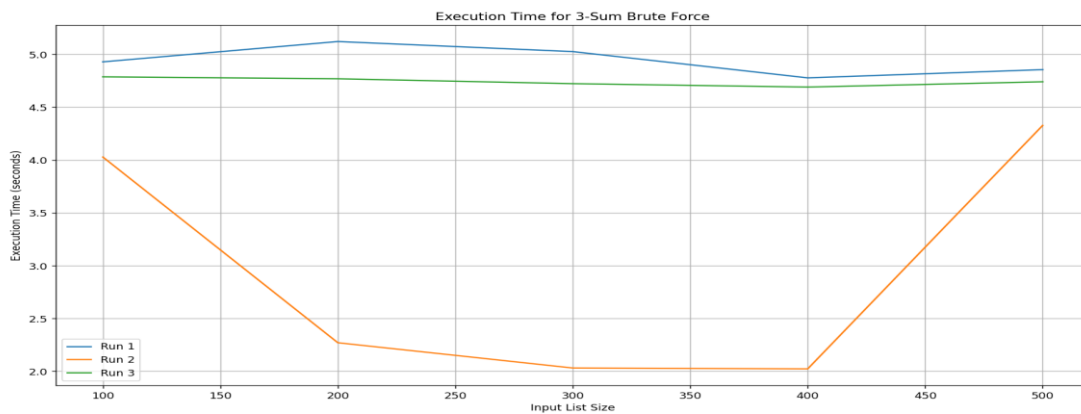
Alla experiment utfördes på samma dator: en Windows enhet med Core i5-processor och 16 GB RAM. Operativsystemet var Windows 11. För att säkerställa konsekventa resultat minimerades eller stängdes andra applikationer under experimenten för att minska externa störningar.

Varje tidsstämning som presenteras här är resultatet av en enda körning, som upprepas tre gånger för varje indatastorlek för att observera eventuella variationer.

Inmatningsstorleken sträckte sig från 100 till 500 i steg om 100, vilket valdes för att producera exekveringstider inom ett lämpligt intervall för visualisering och analys, med målet att hålla körningstiderna mellan 0,001 och 5 sekunder.

De indata som användes för 3-sum var en lista över en slumpmässiga heltal av storlek sz , där varje heltal inom intervallet $[-10 * sz, 10 * sz]$. Målsumman var alltid satt till 0. Vilket innebär att algoritmen försökte hitta unika tripletter (n_i, n_j, n_k) så att $n_i + n_j + n_k = 0$.

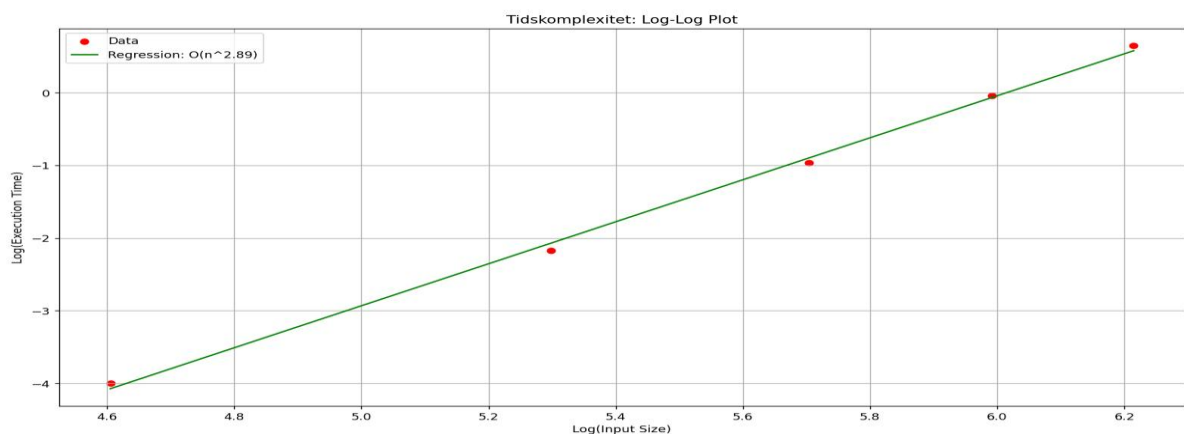
Tids mätningarna plottades för att visa hur exekveringstiden för brute-force algoritmen skalade med ökande indatastorlekar. Resultaten från dessa experiment visade på en ökning i exekveringstid i takt med att storleken på indata växte, vilket var förväntat från en algoritm med $O(n^3)$ - tidskomplexitet.



Grafen visar att exekveringstiden ökar kraftigt med större inmatningsstorlek, vilket är i linje med algoritmens teoretiska tidskomplexitet $O(n^3)$.

Mindre skillnader observeras mellan de tre körningarna, vilket indikerar att algoritmens prestanda främst beror på inmatningsstorleken snarare än specifika slumpdata.

Algoritmen är opraktisk för stora inmatningsstorlekar på grund av dess kubiska tillväxt och bör optimeras för att hantera större datasätt.



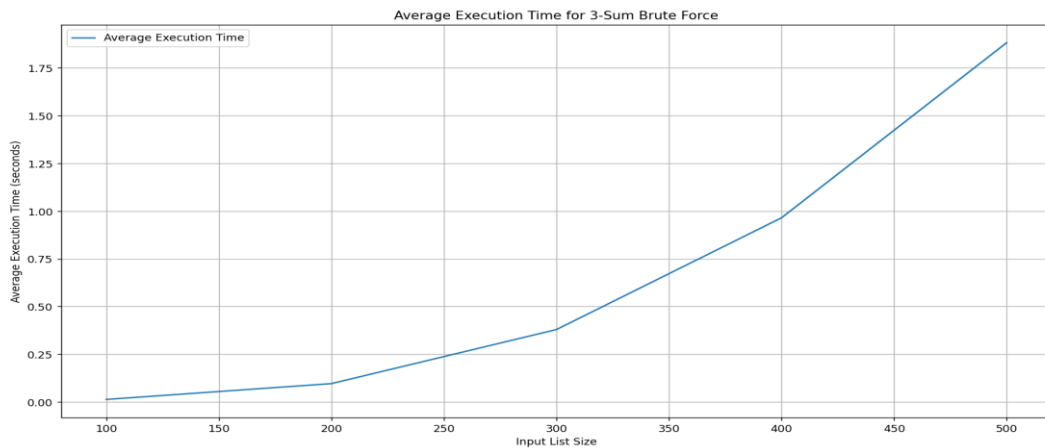
Tidskomplexitet:

För att analysera tidskomplexiteten för 3-sum Brute Force – algoritmen användes log-log plot och linjär regression. Algoritmen testades med olika inmatningsstorlekar (från 100 till 500) och för varje inmatningsstorlek beräknas den genomsnittliga exekveringstiden över fem körningar. Dessa exekveringstider användes för att uppskatta exponenten k i tidskomplexiteten $O(n^k)$.

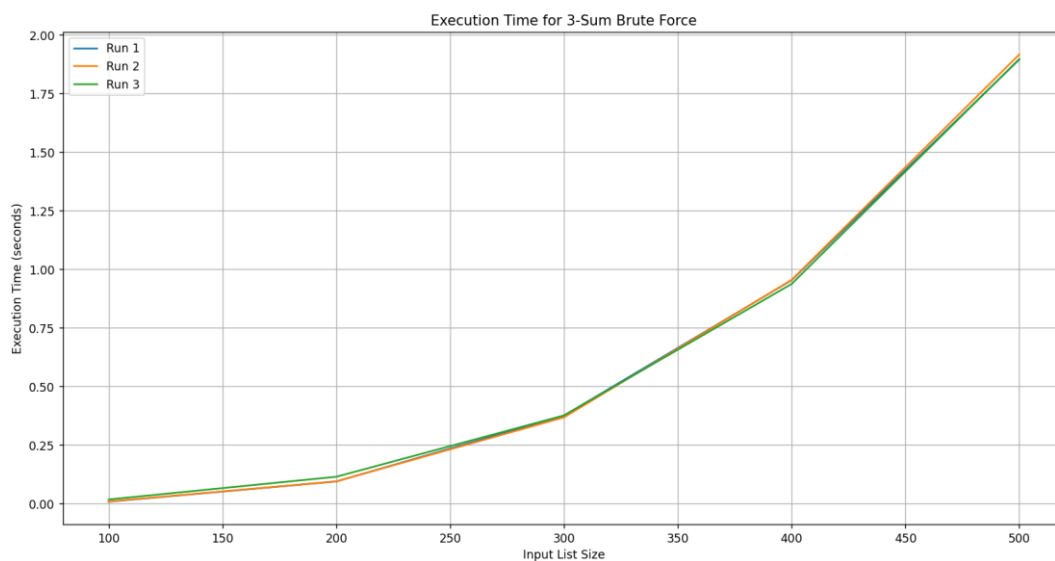
För att analysera förhållandet mellan inmatningsstorlek och exekvering tid, omvandlas både inmatningsstorlekar och exekveringstider till logaritmiska skalor. Genom att applicera linjär regression på den log-log skalan, beräknas lutningen på den resulterande linjen. Lutningen gav uppskattningen av exponenten k .

Resultaten visade att den beräknade tidskomplexiteten var $O(n^{2.91})$, vilket innebär att algoritmen visar en nästan kubisk tillväxt, men något lägre än den teoretiska $O(n^3)$.

Denna uppskattning bekräftar att algoritmen har en exponentiell tillväxt, men med en mindre brant ökning än väntat enligt den teoretiska tidskomplexiteten. I den log-log plotten ses data som röda punkter, och den gröna linjen representerar den linjära regressionen som visar den uppskattade tidskomplexiteten. Grafen illustrerar hur exekveringstiden växer med ökande inmatningsstorlek och ger en visuell representation av den empiriska tidskomplexiteten för algoritmen.



Grafen bör visa att exekverings tiden växer snabbt när inmatningsstorleken ölar, vilket är typiskt för algoritm med en tidskomplexitet av $O(n^3)$. Detta innebär att när storleken på listorna ökar, krävs exponentiellt mer tid för att bearbeta dem. Om man fortsätter att öka storleken på inmatningarna ytterligare kan exekveringstiden bli opraktiskt lång, vilket innebär att Brute Force algoritmen inte är effektiv för större datamängder.



Grafen visar sambandet mellan inmatningsstorlek och exekveringstid för 3-Sum Brute Force algoritmen. På x axeln visas de logaritmerade inmatningsstorlekarna ($\log(\text{Input Size})$) och på y-axeln visas de logaritmerade exekveringstiderna ($\log(\text{Execution Time})$)

De data som används för att rita grafen representeras av gröna linjen, som är resultatet av en linjär regression. Linjen illustrerar hur exekveringstiden växer när inmatningsstorleken ökar, vilket ger en indikation på tidskomplexiteten hos algoritmen. Regressionen ger en uppskattning av tidskomplexiteten i formen $O(n^k)$, där k är lutningen.